

Collie: Token-Level Co-Serving of LLM Inference and Fine-Tuning with SLO Guarantees

NSDI'26 Submission #183, 12 pages

Finetuning large language models (LLMs) is essential for task adaptation, yet today's serving stacks isolate inference and finetuning on separate GPU clusters—wasting resources and underutilizing hardware. We introduce Collie, the first system to *co-serve* LLM inference and PEFT-based finetuning on shared GPUs by fusing computation at the token level. Collie's static compilation optimizations—*dependent parallelization* and *graph pruning* significantly shrink activation memory, leading to end-to-end GPU memory savings by up to 80%. At runtime, a novel *token-level finetuning* mechanism paired with a hybrid token scheduler dynamically interleaves inference and training tokens within each co-serving iteration, meeting strict latency SLOs while maximizing utilization. In end-to-end benchmarks on LLaMA-3.1-8B, Qwen-2.5-14B, and Qwen-2.5-32B, Collie sustains the inference SLO requirements up to 20 req/s, and improves finetuning throughput by $1.9 - 4.8\times$ under heavy inference workloads and $2.5 - 6.8\times$ under light loads, preserving over 76% of peak finetuning progress even at peak demand.

1 Introduction

Recent advancements in large language models (LLMs) such as GPT [12, 24, 57], PaLM [17], and LLaMA [73] have shown strong capabilities of generating natural language texts across various application domains [23, 47, 68, 88]. Given the increasingly high cost of LLM pre-training, there is a trend of standardizing foundational pre-trained LLMs and sharing sub-models across multiple downstream tasks [34, 86] through finetuning (i.e., continuous training on small task-specific datasets). For example, a series of *parameter-efficient finetuning* (PEFT) techniques [32, 34, 35, 41, 43, 59] have been proposed to update a subset of trainable parameters from an LLM or introduce a small number of new parameters into the LLM while keeping the vast majority of the original LLM parameters frozen. The base LLM sharing paradigm provides opportunities to multiplex resources and improve cluster utilization (e.g., Google [11], Microsoft [35], and Tencent [56]). Many recent efforts [13, 36, 66, 80, 91, 97] inspired by these opportunities to build multi-task serving systems, handling inference requests for multiple fine-tuned LLMs simultaneously in a single GPU cluster.

Another undeniable trend is the increasing importance of LLM finetuning¹. Almost all LLM serving companies, including OpenAI GPT-4o [58], Google Gemini [28], and Databricks Mosaic AI [19], now offer commercial finetuning APIs to facilitate the creation of custom models. This trend is not only due to the widespread adoption of LLMs but also driven by the growing demand for task-specific model cus-

tomization [49], data access control [69], and the need for continuous and frequent model updating [26]. Unlike inference tasks with strict latency requirements (e.g., microseconds to milliseconds per token), finetuning tasks usually allow for greater latency tolerance with coarser time budget granularity (e.g., minutes to hours). As such, the current practice involves operating these two tasks on *dedicated* clusters [14, 27, 78], which exclusively use hardware resources through specialized systems optimized for latency and throughput, respectively.

However, real-world LLM inference workloads usually exhibit fluctuating and bursty request arrival patterns [77]. Serving inference requests must meet latency-based service level objectives (SLOs), which may vary from stringent to relaxed depending on the application scenario. To avoid SLO violations, service providers must either over-provision GPU resources or dynamically adjust resource allocation. The former strategy can lead to significant computational waste, while the latter introduces substantial overhead due to the need for frequent redeployments and process switching [10, 54]. In this paper, we explore a key research question: *Can we design a system that simultaneously serves both inference and finetuning tasks within a shared GPU cluster, effectively accommodating the time-varying inference computational demands?*

We introduce Collie, the first co-serving system for LLM inference and finetuning. Collie introduces a *PEFT-as-a-service* (PaaS) interface, which unifies inference and finetuning tasks, enabling their joint execution on shared GPU resources. The key insight behind Collie is that inference and finetuning tasks, when using the same base LLMs, can be merged rather than executed separately.

This co-serving approach presents multiple advantages. First, since the base LLM is generally frozen during finetuning, the base model parameters can be stored without redundancy, therefore reducing the memory footprint. Second, when faced with bursty inference demands, Collie can adaptively reassign GPU resources by temporarily reducing the proportion of finetuning tasks, which inherently possess higher latency tolerance. Once the peak inference workload has passed, the finetuning tasks can reclaim resources, thereby optimizing overall throughput and maximizing GPU utilization without compromising inference SLOs. This dynamic strategy provides a novel avenue for cloud LLM service providers to improve operational cost-effectiveness and service capabilities.

Beyond supervised PEFT, a growing class of reinforcement-learning-based fine-tuning algorithms – most notably Group Relative Policy Optimization (GRPO) used in DeepSeek-R1 [31], and multi-objective variants thereof [44] – require the model to generate complete sequences on-the-fly in or-

¹For simplicity, we use the term finetuning to refer to PEFT for the rest of the paper.

der to compute policy gradients or reward signals. In each RLHF [65] or GRPO update, the LLM must sample continuations (i.e., perform inference), score them against a learned or hand-crafted reward, and then backpropagate through those same tokens. Because inference and policy-gradient steps are interleaved at the token level, splitting inference and training onto separate GPUs incurs costly data movement and context-switching. This tight coupling further motivates Collie’s token-level co-serving, which fuses generation and gradient updates in a single pass while still meeting stringent inference SLOs.

Challenges. Co-serving inference and finetuning tasks within a single system, as envisioned by Collie, presents significant challenges due to their differing workload characteristics. Collie must address three key challenges.

First, *how can inference and finetuning tasks be co-located while minimizing the memory footprint?* Traditional multi-tenancy ML scheduling systems share resources across tasks and models [81], but suffer from the high memory demands associated with LLMs. Furthermore, existing finetuning systems usually retain all intermediate activations similar to training systems to enable gradient computation during backpropagation, which substantially increases their resource consumption compared to pure inference tasks. This discrepancy makes it challenging to manage system resources between finetuning and inference tasks.

To address this challenge, Collie co-locates finetuning tasks with inference tasks by enforcing them to share the base LLM parameters and intermediate activation buffers. In scenarios where the base LLM has already been deployed in a distributed system, Collie uses *dependent parallelization* to optimize the placement of small, trainable *bypass networks* (e.g., LoRA adapters [35]) introduced during finetuning. This is achieved by solving a constrained optimization problem detailed in §5.1.

Unlike existing systems that preserve all intermediate activations throughout a forward pass for finetuning, Collie leverages the observation that most intermediate activations are only necessary for computing gradients of *frozen parameters*. By implementing *graph pruning*, Collie eliminates unnecessary data dependencies associated with these frozen parameters (see §5.2). These memory optimization strategies allow Collie to reduce the memory overhead associated for finetuning by up to $8\times$ compared to existing systems.

Next, *how can we run inference and finetuning tasks simultaneously while minimizing the interference with inference latency and SLO attainment?* Previous work on traditional ML workloads [62] has shown that co-locating inference with training can significantly degrade inference performance. Unlike conventional training workloads that require coarse-grained temporal or spatial resource sharing [15], LLM finetuning provides a unique opportunity to batch base LLM computation and parameter access across requests from different finetuned variants [80]. However, due to differences

in execution logic—auto-regressive decoding in inference versus backpropagation in finetuning—merging the two types of tasks remains difficult.

To overcome this challenge, Collie introduces a novel *token-level finetuning* mechanism, which decomposes the finetuning computation of a sequence into smaller, token-level units (§6.1). During the forward pass, finetuning tokens follow the same operator execution order as decoding tokens in inference requests. These tokens are then processed using fused GPU kernels. For the backward pass, Collie launches separate GPU streams for finetuning tokens and adopts a layer-wise execution strategy to minimize memory usage. This design significantly reduces the performance impact of finetuning tokens on inference latency, enabling efficient co-serving of inference and finetuning tasks on shared hardware.

Finally, *how can we handle fluctuating inference workloads while preserving their latency SLO?* Given the dynamic nature of inference request arrival patterns, provisioning for peak demand often results in significant resource under-utilization. Collie’s fine-grained, token-level execution strategy enables quick adaptation to load fluctuations. Inspired by prior work on co-locating multiple cloud workloads [25], Collie prioritizes inference tokens to meet strict latency constraints, while inserting finetuning tokens in a best-effort manner.

To achieve this, Collie introduces a *hybrid token scheduler*, which coordinates the scheduling of inference and finetuning tokens at each co-serving iteration. This scheduler aims to maximize overall GPU utilization while guaranteeing that inference tasks consistently meet their SLOs (Section 6.2).

Evaluation. We compare Collie’s co-serving approach with existing approaches that employ separate GPUs to perform inference and finetuning and show that Collie achieves significantly higher finetuning throughput. For example, under heavy inference workloads where existing systems must dedicate all available GPUs for inference tasks and therefore cannot make any finetuning progress, Collie can still preserve more than 80% of peak finetuning throughput.

2 Background and Challenges

2.1 Parameter-Efficient Finetuning

Finetuning enables LLMs to adapt to specific domains and downstream tasks [12, 20, 61]. However, full finetuning of an LLM introduces significant computational and memory overheads due to the model’s large parameter count. To address this issue, a class of *parameter-efficient finetuning* (PEFT) methods [22] has been recently proposed. Inspired by the sensitivity of LLMs to input prompts [63], a series of studies introduce trainable prompt embeddings—either prepended to the input sequence or incorporated into attention components—while keeping the original LLM parameters frozen [41, 43, 48]. Recent work also proposed a variety of *adapter* modules to insert lightweight task-specific modules into the Transformer layers. For example, LoRA [35] leverages the low-rank de-

composition concept to construct trainable parameters inserted into the original LLM weights. (IA)³ [46] proposes to scale the pre-trained model weights of an LLM with a trainable vector. Recently, there has been a surge in the proposal of unified approaches that amalgamate various PEFT methods by leveraging human heuristics [32] or employing neural architecture search [51, 96, 98]. Existing PEFT approaches focus on optimizing model performance while minimizing trainable parameters. However, reducing the number of trainable parameters does not inherently imply a corresponding reduction in memory footprint.

2.2 PEFT-based LLM Inference

LLM inference is increasingly essential in both research and industrial applications. Recent work has introduced a diversity of systems for simultaneously serving multiple PEFT-based LLMs. PetS [97] is the first unified framework for multi-task parameter-efficient Transformers serving, which decouples linear layers in the computation graph into task-agnostic shared operations and task-specific PET operations, then optimizes the operator scheduling strategy to improve concurrency and throughput. While PetS only supports early encoder-only models, recent work also focuses on optimizations for generative decoding models. Punica [13] is a system to serve multi-tenant LoRA in a shared GPU cluster. It proposes a new CUDA kernel called Segmented Gather Matrix-Vector Multiplication, which allows batching GPU operations for the concurrent execution of multiple, different LoRA models. S-LoRA [66] is optimized on the kernel introduced by Punica. It proposes unified paging, heterogeneous batching, and a novel tensor parallel algorithm to serve thousands of LoRA components on a single machine. dLoRA [80] further improves efficiency by dynamically adjusting the LoRA batching plans under imbalanced workloads.

2.3 Reinforcement-Learning Based Finetuning

While PEFT methods such as LoRA and adapters optimize a small set of task parameters via supervised losses, recent work has demonstrated that pure reinforcement learning (RL) can substantially enhance LLM reasoning and alignment. In RLHF (e.g., [67]), the model samples candidate completions, a human or reward model ranks them, and policy gradients adjust the generator to favor preferred outputs. More recently, Group Relative Policy Optimization (GRPO) forgoes a separate value network by scoring *groups* of sampled continuations to estimate baselines, reducing critic overhead [31]. Extensions such as multi-objective GRPO further handle safety and helpfulness rewards within a single RL loop [44], and open-source systems like DAPO illustrate scaling these methods to 32B-parameter models [85].

Crucially, all of these RL-based pipelines require *on-the-fly* LLM generations as part of the reward calculation, interleaving inference kernels with backward passes at sub-sequence granularity. This pattern conflicts with the traditional separa-

tion of “inference-only” versus “training-only” clusters, leading to GPU underutilization or high-latency context switches. Collie’s token-level fusion directly addresses this challenge by co-scheduling generation and gradient updates within a single GPU pipeline, while still meeting strict inference SLOs.

3 Co-serving LLM Inference and PEFT

Previous studies have proposed different methods for scheduling heterogeneous workloads (in our case, inference and PEFT) on shared GPU resources. These methods can be classified into three categories: *resource isolation*, *temporal sharing*, and *spatial sharing*. This section discusses the limitations inherent to these conventional approaches when applied to co-serving LLM inference and PEFT. It also highlights the fundamental distinctions between Collie’s co-serving solution and previous work. Figure 1 depicts the GPU execution timelines associated with different scheduling approaches for managing inference and finetuning tasks.

Resource isolation. A common approach to support both inference and finetuning tasks is *resource isolation*, which partitions the available GPUs into two distinct groups, each dedicated to handling one type of workload (see Figure 1(a)). This approach ensures that throughput-intensive finetuning tasks do not interfere with latency-critical inference tasks. When finetuning an LLM, a dataset of requests is provided to train the PEFT layers, with all requests submitted to Collie simultaneously, making it easier to build larger batches that can be processed sequentially. In contrast, inference requests are submitted independently by users who expect timely processing within predetermined SLOs, with the arrival times of these requests unpredictable.

Resource isolation can introduce significant GPU underutilization, particularly in scenarios with variable request arrival rates. This is because finetuning and inference requests cannot be batched together if the number of pending requests of either type is insufficient to fully utilize a batch. With fewer GPUs available to parallelize inference computations, it may not be possible to serve requests fast enough to meet their SLO (e.g., request r_3 in Figure 1(a)).

Temporal sharing. Compared to resource isolation, *temporal sharing* [81, 82] improves GPU utilization by allowing different tasks to share GPU resources through time slices of different lengths depending on various parameters. This method allows both inference and finetuning requests to access all GPUs within the cluster in turns, as shown in Figure 1(b). By using all GPUs to parallelize computation (e.g., with data parallelism or tensor model parallelism), the latency for each request can be significantly reduced, thus aiding in meeting inference SLOs. For example, while inference request r_3 fails to complete in the scenario illustrated in Figure 1(a) due to resource limitations, it successfully executes in Figure 1(b) by leveraging more resources. However, the arrangement of time slices can also be harmful because of the unpredictable

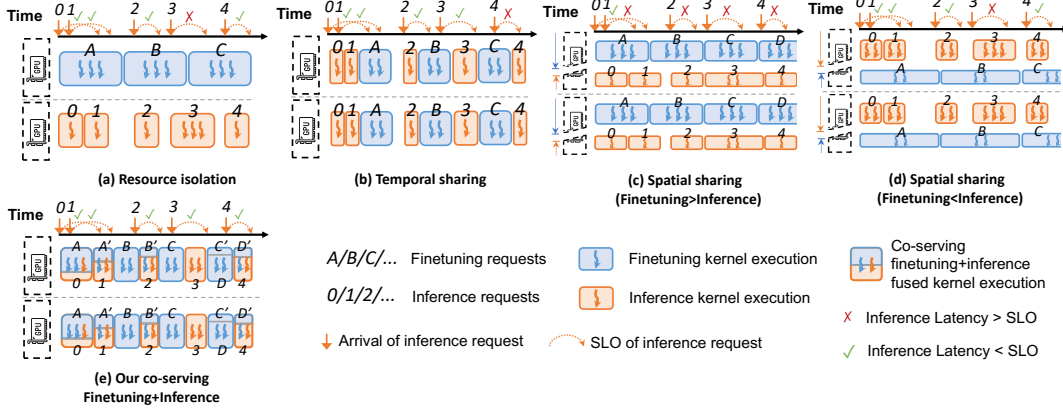


Figure 1: Comparing different resource sharing approaches for serving finetuning and inference. For spatial sharing and co-serving, the height of rounded rectangles illustrates the splitting ratio of GPU resources (e.g., streaming multi-processors).

arrival times of inference requests, which may have a stringent latency requirement that cannot be met (e.g., r_4 has been delayed in Figure 1(b) by pre-scheduled finetuning requests).

Spatial sharing. *Spatial sharing* offers an alternative by enabling simultaneous execution of different tasks at the kernel level, where each task utilizes a specific portion of the GPU resources (i.e., streaming multi-processors). This can be implemented on NVIDIA GPUs through multi-stream programming or by utilizing the Multi-Process Service (MPS) [4]. Furthermore, on the latest Hopper and Ampere architectures, spatial sharing benefits from the Multi-Instance GPU (MIG) [3], which allows for dynamic adjustments of GPU partition configurations. However, these adjustments often lack the flexibility and efficiency required to handle dynamic inference workloads effectively [2, 21, 83]. Figures 1(c) and (d) show two scenarios where a substantial portion of GPU resources is allocated to either finetuning or inference tasks. While over-provisioning for inference tasks may appear to provide better SLO guarantees, it still falls short during peak demand periods such as the unexpected urgent request r_3 . Additionally, over-provisioning wastes resources and significantly slows down the finetuning process. Therefore, relying solely on spatial sharing to manage finetuning and inference tasks often results in suboptimal performance [15, 92].

Collie’s co-serving approach. As shown in Figure 1(e), Collie employs a co-serving approach that uses a fine-grained scheduling mechanism to adaptively manage both inference and finetuning requests. During each co-serving iteration, Collie jointly serves inference and finetuning requests by (1) computing keys and values for inference requests in the pre-filling phase, (2) generating a new token for inference requests in the decoding phase, and (3) performing forward and/or backward passes for selected tokens from finetuning requests.

Unlike temporal or spatial sharing approaches, which schedule GPU kernels of existing systems independently, Collie

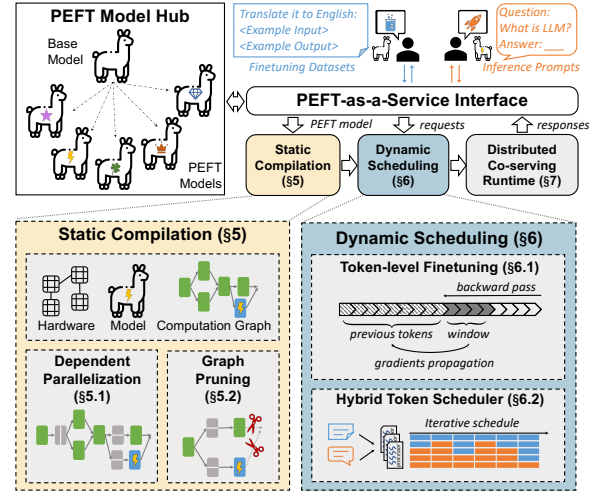


Figure 2: An overview of Collie.

integrates inference and finetuning kernels. This integration reduces the overhead associated with kernel launches and minimizes accesses to GPU device memory to retrieve model weights. In each iteration, Collie dynamically adjusts the allocation of inference and finetuning tokens, strategically balancing the need to meet the SLOs of inference requests with the goal of maximizing GPU utilization. This adaptive approach ensures that Collie adheres to the requirements of inference requests while optimizing the use of available GPU resources.

4 System Overview

Figure 2 shows an overview of Collie, which co-serves both PEFT and inference requests for LLMs. Users can choose to use either finetuning or inference services on a model from the PEFT *model hub*, which stores the backbone LLM and all finetuned variants. The programming interface for both

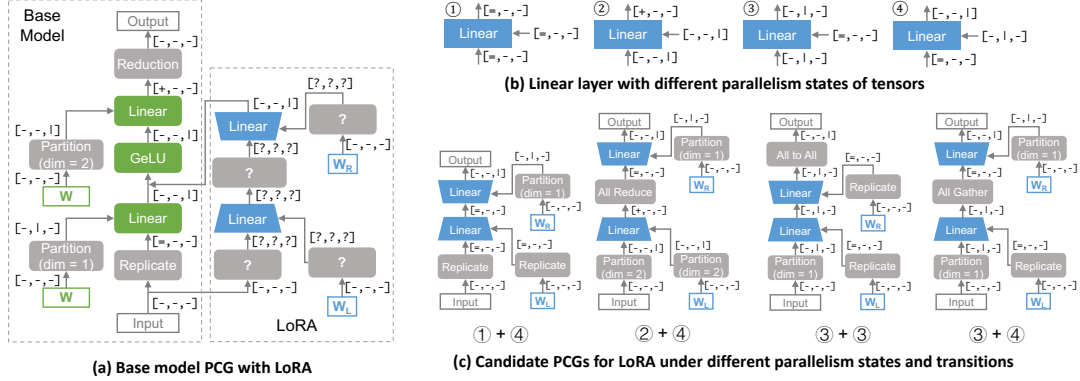


Figure 4: Illustration of Collie’s different dependent parallelization strategies with the LoRA example. Each green (or gray) box indicates a compute (or parallelization) operator, and each edge between operators represents a parallel tensor, and the parallelization states of the tensor’s dimensions are shown next to the edge.

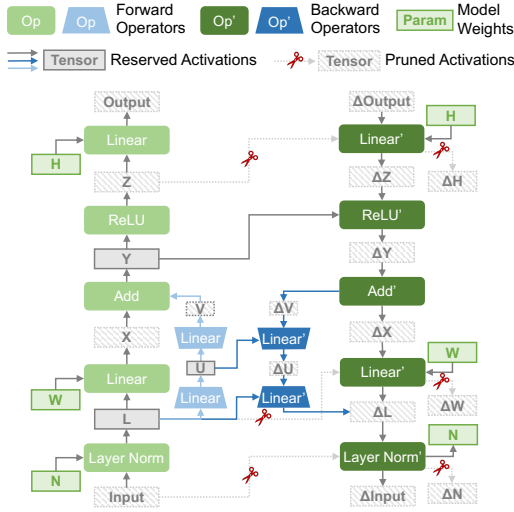


Figure 5: Static graph pruning for an MLP model with LoRA.

of PEFT, the backbone LLM parameters are frozen, and only the parameters in the bypass networks are updated. This insight allows Collie to eliminate most intermediate activations, retaining only those necessary for computing gradients of the bypass networks. Second, different PEFT methods connect bypass networks at varying locations within the backbone LLMs, and therefore require different subsets of intermediate activations for backpropagation. Figure 6 illustrates examples of how various PEFT methods attach their bypass networks to the base LLM.

Building on these observations, Collie performs *static* graph pruning during the construction of the PCG for each registered PEFT model, and *dynamical* scheduling to manage finetuning and inference tasks, along with corresponding memory allocations (see Section 6). Algorithm 1 shows Collie’s graph pruning algorithm. Given a PEFT model’s PCG $\mathcal{G}$, the algorithm generates an execution plan that specifies

whether each intermediate activation should be cached during the forward pass to enable efficient backpropagation.

First, REVERSEAUTODIFF constructs the backward graph $\bar{\mathcal{G}}$ using reverse-mode automatic differentiation [33]. Since all weights of the base LLM are frozen during finetuning, Collie prunes gradients the computation of their weights. Pruning the output tensors of an operator may, in turn, create opportunities to prune downstream operators. Collie maintains a queue Q of operators whose input and/or output tensors have changed. It iteratively processes this queue to discover further pruning opportunities. Upon completion, Collie identifies a set of tensors $\mathcal{A}$ representing the minimal set of intermediate activations that must be preserved for the backward pass. Collie further combines graph pruning with two established memory optimizations: *rematerialization* and *activation compression*.

Rematerialization. Tensor rematerialization [37] consists of selectively discarding some tensors in the forward pass and recomputing them during backward propagation. Originally designed to reduce the peak memory requirement of DNN training, this technique is also beneficial for LLM co-serving. For each remaining tensor $t \in \mathcal{A}$ after graph pruning, Collie rematerializes t if and only if (1) all input tensors of t are stored in $\mathcal{A}$ and (2) recomputing t using the input tensors introduces a low rematerialization overhead.

Activation compression. In addition, Collie also opportunistically applies *lossless* compression for remaining tensors whenever possible. This is because performing backward computation for some operators does not require access to the original input tensors. One example is the ReLU operator $y = \text{ReLU}(x) = \max(x, 0)$ whose derivative is $\partial y / \partial x = 1$ for $x > 0$ and 0 for $x \leq 0$. Therefore, instead of storing the original input tensor x , Collie keeps the bitmask of x .

6 Dynamic Scheduling

Once static compilation produces an executable generalized PCG for a PEFT model, Collie must determine an execution

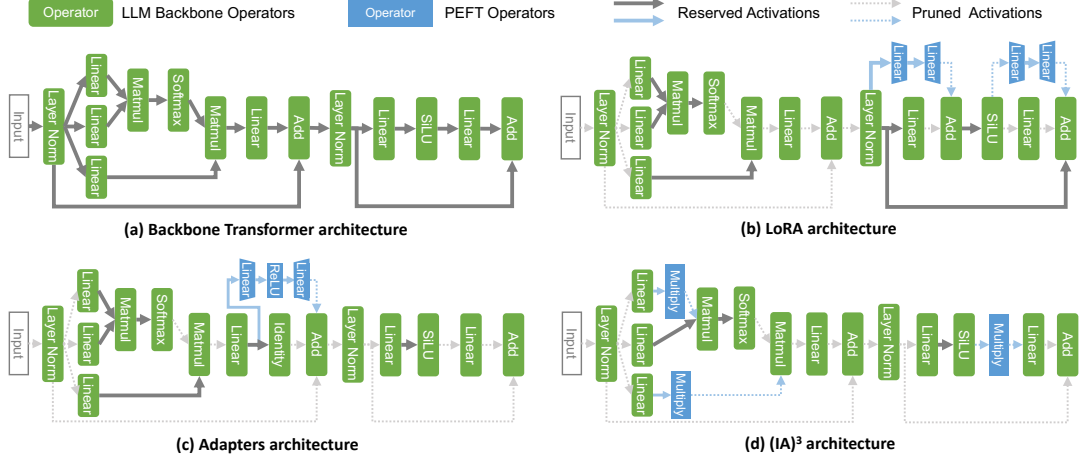


Figure 6: An overview of existing parameter-efficient fine-tuning (PEFT) methods. Green boxes show the operators of the backbone LLM, while blue boxes are the operators introduced by different PEFT methods. Arrows represent intermediate activations to be reserved and the dashed arrows demonstrate they are pruned by Collie.

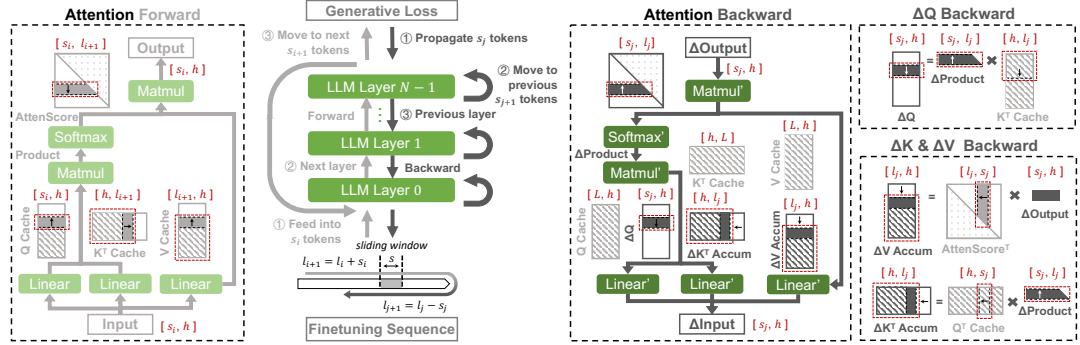


Figure 7: Illustration of attention module's forward and backward execution with Collie's token-level finetuning mechanism.

schedule capable of concurrently handling both finetuning and inference requests. This is particularly challenging due to the distinct characteristics of these two workloads. First, inference requests arrive in an online and highly dynamic manner, requiring the system to rapidly adapt GPU resource allocation to maintain low-latency responses. In contrast, finetuning involves both a forward pass—similar to inference—and a backward pass to update model parameters. Additionally, auto-regressive inference for generative LLMs proceeds in a token-by-token fashion, while finetuning is typically performed at the sequence level to maximize throughput. To address these challenges, Collie introduces a novel *token-level finetuning* mechanism along with a *hybrid token scheduler*.

6.1 Token-Level Finetuning

Processing a fine-tuning request typically involves performing both forward and backward passes across all model layers for the entire input sequence. When co-served with inference requests, this sequence-level execution can significantly interfere with the token-wise inference kernels and degrade

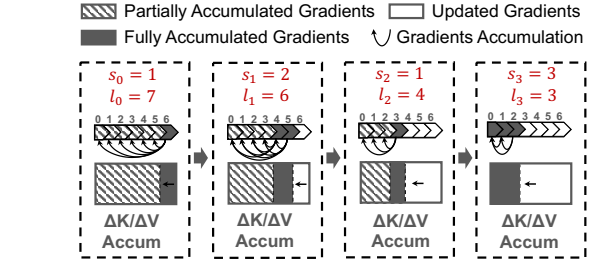


Figure 8: Illustration of KV gradients accumulation in the backward pass of Collie's token-level finetuning mechanism, where s_j and l_j are the window length and the starting position of the j -th slice of the sequence.

system goodput—this is, the effective inference throughput that meets latency SLOs.

To address this problem and maximize GPU utilization, Collie introduces a *token-level finetuning* mechanism, detailed in Algorithm 2. The key idea is to decompose the finetuning

Algorithm 1 Static graph pruning. For an operator n , $\text{UPDATEINPUT}(n, O(n))$ returns a set of input tensors needed in order only to compute $O(n)$ of the operator.

```

1: Inputs: PCG of a PEFT model  $\mathcal{G}$ 
2: Outputs:  $\mathcal{A}$  is a set of tensors to be memorized,  $\mathcal{R}$  is a
   set of tensors to be rematerialized
   ▷ Step 1: computation graph pruning
3:  $\bar{\mathcal{G}} = \text{REVERSEAUTODIFF}(\mathcal{G})$ 
4:  $Q = \emptyset$  ▷  $Q$  is a queue of updated operators
5: for operator  $n \in \bar{\mathcal{G}}$  do
6:   for output tensor  $t \in O(n)$  do
7:     if  $t$  is the weight gradient of the base LLM then
8:        $O(n) = O(n) \setminus \{t\}$ 
9:        $I(n) = \text{UPDATEINPUT}(n, O(n))$ 
10:       $Q.\text{push\_back}(n)$ 
11: while  $Q$  is not empty do
12:    $n = Q.\text{pop\_front}()$ 
13:   for output tensor  $t \in O(n)$  do
14:     if  $\nexists u.t \in I(u)$  then
15:        $O(n) = O(n) \setminus \{t\}$ 
16:        $I(n) = \text{UPDATEINPUT}(n, O(n))$ 
17:        $Q.\text{push\_back}(n)$ 
18:  $\mathcal{A} = \emptyset$ 
19: for operator  $n \in \mathcal{G}$  do
20:   for tensor  $t \in O(n)$  do
21:     if  $\exists u \in \bar{\mathcal{G}}.t \in I(u)$  then
22:        $\mathcal{A} = \mathcal{A} \cup \{t\}$ 
   ▷ Step 2: opportunistically rematerializing tensors
23: for tensor  $t \in \mathcal{A}$  do
24:   Let  $n$  be the operator that outputs  $t$  (i.e.,  $t \in O(n)$ )
25:   if  $I(n) \subseteq \mathcal{A}$  and  $\text{COST}(n) < \text{threshold}$  then
26:      $\mathcal{A} = \mathcal{A} \setminus \{t\}$ ,  $\mathcal{R} = \mathcal{R} \cup \{t\}$ 
27: return  $\mathcal{A}, \mathcal{R}$ 

```

computation into smaller steps using a dynamic sliding window over tokens. This window spans both the forward and backward passes, with its size dynamically determined by the hybrid token scheduler (line 4 and 15) to ensure that inference requests continue to meet their SLO constraints (§6.2).

During the forward pass (lines 3-11), the finetuning sequence is partitioned into windows of s_i tokens (line 5), which are incrementally fed into the model. The model processes each token window layer by layer (lines 7-8), computes the generative loss for the window (line 10), and then advances to the next set of tokens (line 11). To preserve the semantic of full sequence-level finetuning, Collie caches key and value tensors—similar to incremental decoding in inference [84]—as well as query tensors, which are reused during backward attention computations. This caching follows the *causal mask* constraint inherent in LLMs.

During the backward pass (lines 12-21), Collie executes the model layers in reverse order (line 13). Within each layer, the

Algorithm 2 Token-level finetuning mechanism.

```

1: Inputs: PEFT model  $M$  depth of  $N$ ; Fine-tuning request
    $r$  with sequence length of  $L$ 
2: Outputs: PEFT model gradients  $\Delta M$ 
3: for  $i \leftarrow 0, l_0 \leftarrow 0; l_i < L; i++$  do ▷ Forward pass
4:    $s_i \leftarrow \text{HYBRIDTOKENSCHEDULER}(l_i, L)$ 
5:    $r_i \leftarrow \text{SLICE}(r, l_i, s_i)$ 
6:    $X_{0,i} \leftarrow \text{TOKENIZE}(r_i)$ 
7:   for model layer index  $n \in \text{RANGE}(0, N)$  do
8:      $X_{n+1,i}, Q_{n,i}, K_{n,i}, V_{n,i} \leftarrow \text{FORWARD}(M_n, X_{n,i})$ 
9:      $QKVCache_n \leftarrow \text{APPEND}(Q_{n,i}, K_{n,i}, V_{n,i})$ 
10:     $Loss_i \leftarrow \text{GENERATIVELOSS}(X_{N,i}, r_i)$ 
11:     $l_{i+1} \leftarrow l_i + s_i$ 
12:  $Y_N \leftarrow Loss$ 
13: for model layer index  $n \in \text{RANGE}(N-1, -1, -1)$  do
14:   for  $j \leftarrow 0, l_0 \leftarrow L; l_j > 0; j++$  do ▷ Backward pass
15:      $s_j \leftarrow \text{HYBRIDTOKENSCHEDULER}(l_j, 0)$ 
16:      $Y_{n+1,j} \leftarrow \text{SLICE}(Y_{n+1}, l_j, s_j)$ 
17:      $Y_{n,j}, \Delta K_{n,j}, \Delta V_{n,j} \leftarrow \text{BACKWARD}(M_n, Y_{n+1,j},$ 
        $QKVCache_n, \Delta KVAccum_n)$ 
18:      $\Delta KVAccum_n \leftarrow \text{ADD}(\Delta K_{n,j}, \Delta V_{n,j})$ 
19:      $G_{n,j} \leftarrow \text{SLICE}(\Delta KVAccum_n, l_j, s_j)$ 
20:      $\Delta M_n \leftarrow \Delta M_n + \text{CALCULATEGRADS}(M_n, G_{n,j})$ 
21:      $l_{j+1} \leftarrow l_j - s_j$ 
22: return  $\Delta M$ 

```

finetuning request is again divided into token windows (lines 14-16). However, unlike the relatively straightforward forward pass, token-level backward execution of the attention module is more complex due to auto-regressive token dependencies.

We illustrate the forward and backward PCGs of the attention module in Figure 7, with the forward graph on the left and the backward graph on the right. When computing gradients for a slice of s_j tokens, the resulting gradients for the queries, keys, and values—denoted as ΔQ , ΔK , ΔV —have shapes of $[s_j, h]$, where h is the hidden size. To compute ΔQ , the model must access the cached keys from all preceding l_j tokens, consistent with the causal mask in forward pass.

However, due to the autoregressive nature of attention, the loss from the s_j tokens also contributes to gradients with respect to previous tokens. As a result, both ΔK and ΔV have shapes of $[l_j, h]$, encompassing all tokens prior to and including the current window. As shown in Figure 8, Collie accumulates these KV gradients across steps. Only after the full backward traversal of the sequence is complete does Collie apply the accumulated gradients (of size $[s_j, h]$) to compute gradients for subsequent model parameters. Note that, this accumulation strategy only slightly increases memory consumption because Collie’s layer-wise execution design allows each layer to reuse the workspace memory allocated by the previous layer, ensuring efficient memory utilization during backward pass.

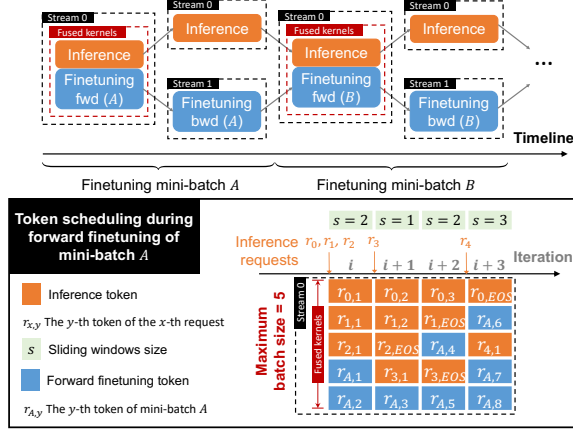


Figure 9: Execution timeline and token scheduling when co-serving inference requests and a finetuning mini-batch.

6.2 Hybrid Token Scheduler

Collie’s *hybrid token scheduler* coordinates token-level GPU execution in two stages. First, Collie determines the scheduling of available inference requests based on specific scheduling policies. By default, Collie adopts Orca’s *iteration-level scheduling* [84], which maintains a fixed maximum batch size and dynamically replaces each completed request with a new one whenever available. To further mitigate blocking caused by long input sequences, Collie incorporates the chunked-prefill optimization [6].

Second, after determining the inference schedule for a given iteration, Collie opportunistically appends as many finetuning tokens as possible—determined by the sliding window size s —to maximize GPU utilization. The number of finetuning tokens added is determined automatically using the formula $s = \arg \max f(c, s) \leq \text{SLO}$, where $f(\cdot, \cdot)$ is the latency estimation function and c is the number of inference tokens scheduled in the current iteration. Here $f(\cdot, \cdot)$ is derived via offline profiling of the LLM’s execution [55]. Such window-based, best-effort scheduling mechanism ensures that Collie can automatically adapt to inference workload fluctuations, maintaining inference SLOs while opportunistically allocating compute to finetuning tasks (§8.3).

Figure 9 illustrates the execution timeline of Collie and shows an example a token scheduling plan for a finetuning mini-batch A, under a representative inference requests arrival pattern (i.e., r_0 to r_4). To preserve the semantics of finetuning, Collie enforces the execution dependencies between the forward and backward passes of each mini-batch. During the forward pass, Collie leverages fused GPU kernels to jointly process both inference and finetuning tokens. This approach avoids additional kernel launch overhead and is enabled by the shared token-wise computation logic between inference and finetuning. In particular, finetuning tokens follow the same execution pattern and causal masking rules as inference tokens during the prefill phase, guaranteeing compatibility

at the kernel level. For the backward pass, Collie adapts a space-sharing approach, using two separate GPU streams: one for inference tokens and another for finetuning tokens. This design allows concurrent execution without interference.

7 Implementation

Collie is built on top of FlexFlow Serve [53] with 9K lines of C++ and CUDA code. It comes with both C++ and Python bindings, and its Python interface is fully compatible with the HuggingFace’s PEFT [50] and transformers [79] libraries. This means that users can easily serve models imported directly from HuggingFace, and even upload finetuned PEFTs to the HuggingFace Hub. Moreover, Collie supports any PEFT models with LLAMA [73], GPT [12], OPT [90], Falcon [9], or MPT [70] backbones. In addition, we provide instructions on how to add support for custom models from HuggingFace.

Memory management. Collie manages the required GPU memory using a combination of static and dynamic allocation. Static allocation reserves GPU space for the backbone weights and the KV cache required for incremental decoding during inference. Dynamic allocation is used for gradients, activations, and optimizer states. When a finetuning request is received, Collie allocates memory during the first forward pass and reuses it during subsequent backward passes, freeing up space when it’s no longer needed.

Key-value gradient accumulator. To support token-level finetuning, it is necessary to accumulate key-value gradients for every preceding token in a given sequence during backpropagation. Collie utilizes static allocation to reserve space for key-value gradient accumulation. During backpropagation, the gradients for keys and values of a specific segment are obtained, and the position of this segment in the overall sequence is determined. Collie then accumulates the new key-value gradients of all subsequently scheduled segments. When a segment is scheduled during backpropagation, it includes accumulated gradients from future tokens.

8 Evaluation

LLMs and PEFT models. We evaluated the performance of Collie by comparing it with existing approaches across three publicly available LLMs: LLaMA-3.1-8B [74], Qwen-2.5-14B [60], and Qwen-2.5-32B [60]. Pre-trained model weights were obtained from their respective HuggingFace repositories [1]. Since existing LLM serving systems only support base models and LoRA-based PEFT, our evaluation focuses on these two configurations to ensure fair comparison. Following prior work, we applied LoRA with a rank of 16 to the down projection layers in each multi-layer perceptron (MLP) block of the models. This setup yields 9.4M, 14.5M, and 25.16M trainable parameters for the three LLMs.

Platform. All experiments were performed on the Perlmutter supercomputer [5]. Each compute node is equipped with an AMD EPYC 7763 64-core 128-thread processor, 256 GB

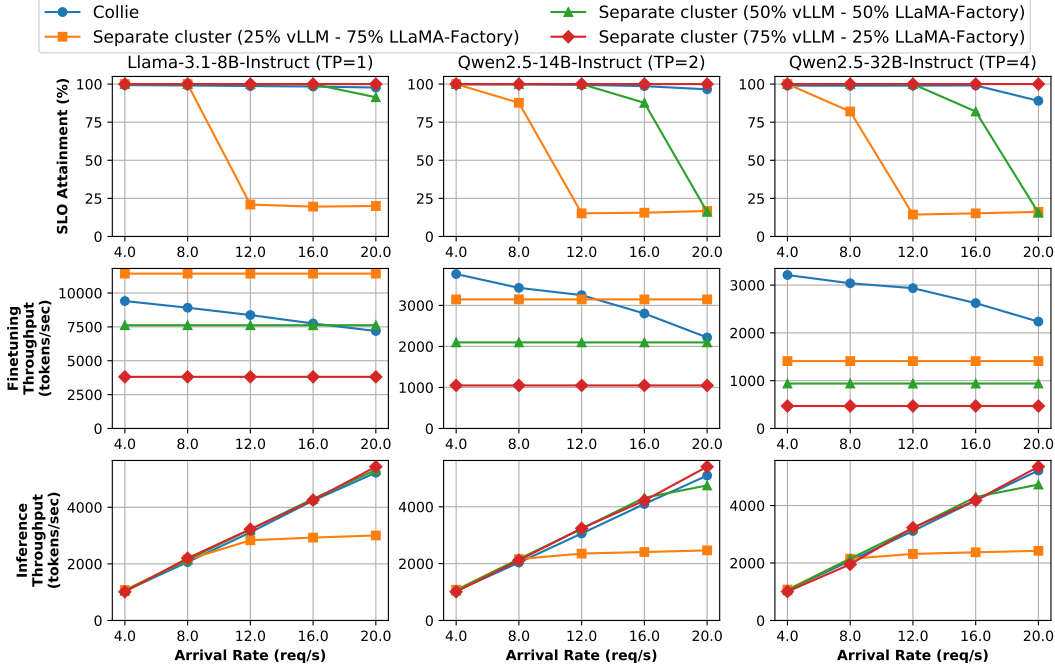


Figure 10: End-to-end comparison between co-serving and using separate resources on three models.

DRAM, and four NVIDIA A100-SXM4-80GB GPUs. Nodes are interconnected via HPE Slingshot with a bandwidth of 200 Gb/s. We employed tensor-model parallelism to serve and finetune the LLMs.

Workload. To synthesize the inference workloads, we sampled the prompt lengths and generation lengths of each request from the ShareGPT [72] dataset, following prior work [7, 39, 95]. However, ShareGPT does not include request arrival times. To address this limitation, we used arrival time data from a recent production trace [76] collected from ChatGPT serving on Azure. The original trace spans multiple days. To evaluate the online serving capabilities of Collie, we sampled 20-minute intervals from the trace and replayed the requests, adjusting the arrival times to simulate different average arrival rates. We used a per-token inference latency of 50ms, 75ms, and 75ms as the SLO for inference requests, following prior work [7]. We set the time-to-first-token (TTFT) SLO to 5s per request to prevent excessive queueing.

We sampled each finetuning request from the NovaSky-AI reasoning dataset [71], and truncate each sequence length to a maximum of up to 8192 tokens. We used the Adam optimizer for training [38].

8.1 End-to-end Comparison

In this section, we compare Collie’s co-serving approach with existing approaches that employ separate clusters for inference and finetuning tasks. In the separate cluster approach, we use vLLM [39] and LlamaFactory [94] as the inference and finetuning systems, respectively, as they represent state-of-

the-art solutions for these workloads. In this experiment, we allocated 4, 8, and 16 A100 GPUs for LLaMA-3.1-8B, Qwen-2.5-14B and Qwen-2.5-32B, respectively, to serve inference and finetuning workloads.

Figure 10 shows the results. The separate cluster approach applied a tensor parallel degree of 1, 2, and 4 for the three models, respectively, creating an overall of four inference and finetuning pipelines. We explored various strategies to distribute resources between inference and finetuning tasks. For example, a configuration of 25% vLLM - 75% LlamaFactory indicates that 25% of the resources (i.e., one pipeline) are dedicated to inference, while 75% (i.e., 3 pipelines) are for finetuning.

Across all three models, configurations using one or two pipelines for inference—25% vLLM and 50% vLLM—are only capable of managing lightweight inference workloads, achieving limited inference throughput and SLO attainment under more intensive workloads. Conversely, dedicating three pipelines to inference (75% vLLM - 25% LlamaFactory) maintains high SLO attainment for inference tasks but results in suboptimal finetuning throughput.

Collie’s co-serving approach maintains near-optimal SLO attainment but without compromising finetuning performance. By prioritizing latency-sensitive inference tasks in each co-serving iteration and opportunistically appending as many finetuning tokens as possible, Collie maximizes GPU utilization and improves finetuning throughput.

Across all three models, Collie matches the 75% vLLM - 25% LlamaFactory configuration in inference SLO attainment

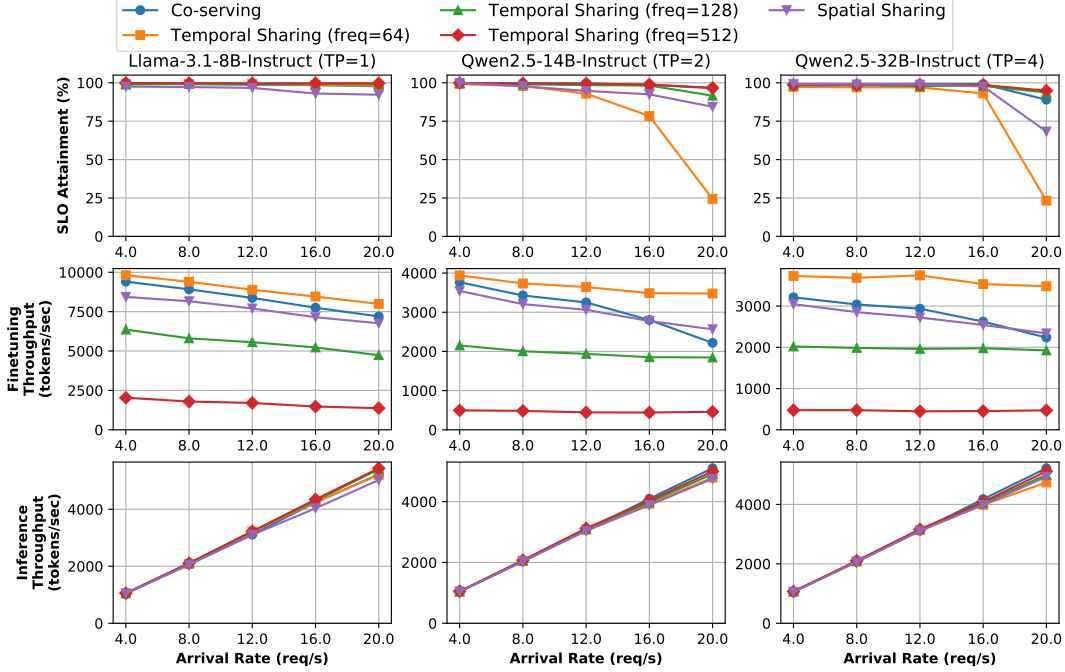


Figure 11: End-to-end comparison between co-serving

(at or above 90% even at 20 req/s) and inference throughput, while dramatically improving finetuning throughput. Specifically, under heavy inference loads (20 req/s), Collie sustains finetuning throughputs of 7.2K, 2.2K and 2.2K tokens/s for LLaMA-3.1-8B, Qwen-2.5-14B, and Qwen-2.5-32B respectively, compared to only 3.8K, 1.0K, and 0.5K tokens/s in the 75% vLLM - 25% LlamaFactory setup—i.e. a $1.9\times$ - $4.8\times$ improvement. Under light inference loads (4.0 req/s), Collie sustains finetuning throughputs of 9.4K, 3.7K, and 3.2K tokens/s for the same models, translating to a $2.5\times$ - $6.8\times$ improvement over the 75% vLLM - 25% LlamaFactory setup.

8.2 GPU Scheduling

This section compares Collie’s co-serving mechanism with two commonly used GPU scheduling strategies: *temporal sharing* and *spatial sharing*. Temporal sharing interleaves the execution of inference and finetuning tasks over time, as illustrated in Figure 1. Spatial sharing simultaneously launches inference and finetuning computations using separate CUDA resources. Both strategies are implemented on top of Collie by replacing the co-serving mechanism with their respective scheduling strategies. To ensure a fair comparison, all memory optimizations are enabled in each baseline.

Figure 11 shows the results. For temporal sharing, interleaving one inference iteration with one finetuning iteration would violate the SLOs for nearly all inference requests. This is because each inference iteration is expected to complete within tens of milliseconds, whereas a finetuning iteration requires several seconds. To mitigate this issue, in the temporal

sharing experiments, we interleave each finetuning iteration with n inference iterations, where n is the inference frequency shown in the figure.

While adopting a low inference frequency in temporal sharing (i.e., frequency = 64) maximizes finetuning throughput, it adversely impacts the SLO attainment and throughput for inference requests. Employing a frequency of 128 enables temporal sharing to match the inference throughput and SLO attainment rate of co-serving. However, it also reduces finetuning throughput by 1.2 - $1.8\times$ compared to co-serving.

Temporal sharing allocates separate GPU resources for inference and finetuning, achieving comparable finetuning throughput to co-serving. However, it remains suboptimal in SLO attainment under heavy inference workloads due to the interference between inference and finetuning tasks.

8.3 Case Study

This section describes a case study of Collie using a 10-min interval of the BurstGPT trace with Qwen-2.5-14B model. To ensure compatibility with our experiment environments, we replayed this trace segment and re-scaled its arrival intensity, as previous works have done [8, 29, 54, 89]. In the experiment, each inference request and inference request are sampled from the ShareGPT dataset and the Sky-T1 dataset respectively.

In our case study, as shown in Figure 12, we observed that the arrival rate of the inference requests initially increased to a peak level after around 90 seconds, and then gradually decreased with some peaks. Collie could automatically detect the fluctuations in the workload and improve the ratio of

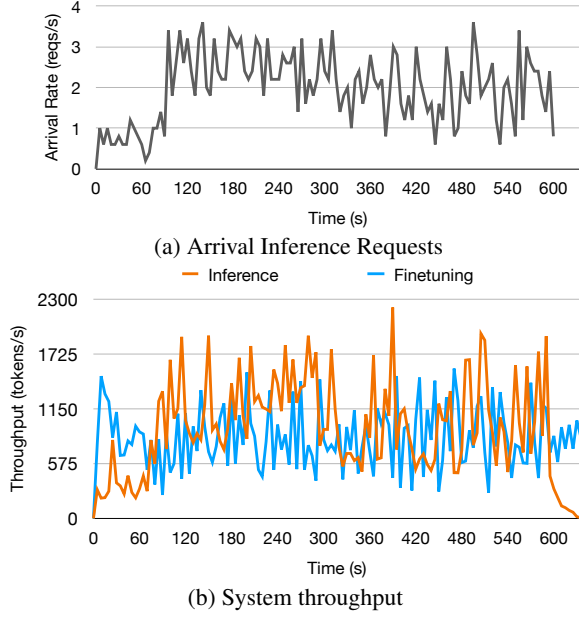


Figure 12: Case study of Collie’s system throughput for fluctuating inference workload.

inference tokens (vs fine-tuning tokens) in each iteration’s batch. This significantly increased inference throughput from a few hundreds to 2.25K.

8.4 Memory Optimization

Collie introduces a series of memory optimizations to reduce the memory overhead of PEFT finetuning and allow co-serving finetuning with inference workloads. To understand the effectiveness of Collie memory optimizations, we perform an ablation study that incrementally turns off these optimizations (graph pruning, rematerialization, and token-level finetuning) and measures the memory requirements. Figure 13 shows the activation memory requirements of Collie for different finetuning methods on a 70B LLM and a sequence length of 1024. Collie saves 85%-87% memory requirements for activations compared to existing approaches. The major improvement comes from graph pruning. With graph pruning only, Collie achieves 71%-74% activation memory overhead reduction. Rematerialization and token-level finetuning further reduce memory overhead by 0-8% and 4%-10%, respectively.

9 Related Work

ML serving and inference systems. Numerous ML serving systems [18, 29, 30, 40, 62] have been proposed in recent years to address various challenges such as latency prediction, system scalability, model performance, swapping [10], preemption [16], workload estimation [89], cost-effectiveness [87], and more. These systems include Nexus [64], which focuses on batching and scheduling multiple small DNN models within a GPU for higher execution throughput, and

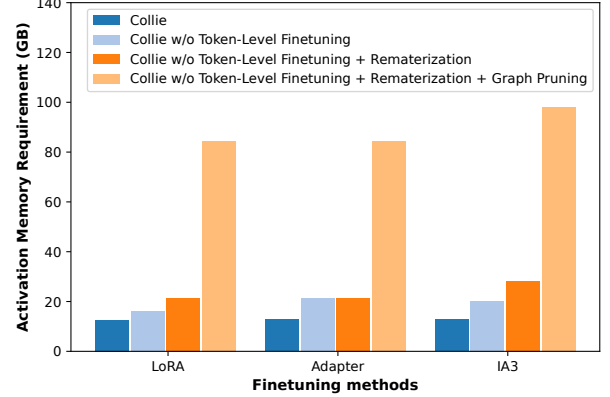


Figure 13: Ablation study of memory optimizations.

Gpulet [15], which improves resource assignment and utilization by creating multiple virtual GPUs within a single physical GPU. However, these systems are mainly designed for small CNN models. Recently, LLM serving systems attracts lots of attention [52], such as vLLM [39], Sarathi [7], and SpecInfer [53]. Among them, AlpaServe [45] extends the temporal sharing approach to LLMs via model parallelism across GPUs. Some concurrent PEFT serving systems like Punica [13] and S-LoRA [66] follow the design of PetS [97] and further improve it by leveraging new GPU kernels for latest PEFT methods like LoRA [35]. However, most of these existing works ignore the resource utilization issue in LLM inference systems. To the best of our knowledge, Collie is the first PEFT co-serving system that concurrently executes finetuning and inference tasks and improves resource utilization without affecting inference latency.

GPU resource sharing for ML. GPU resource sharing is an emerging field of study focusing on the efficient allocation and management of GPU resources among multiple ML users or applications. Lyra [42] allocates dedicated resources for training and serving with dynamic adjustment. Gandiva [81] and Antman [82] take a time-slicing approach to switch between different training jobs for better cluster utilization. GSLICE [21] targets the over- and under-provisioning problems in multi-steam and MPS approaches by developing an adaptive GPU resource allocation and batching scheme for tasks with different SLO constraints. Although these approaches can not directly solve the resource utilization problem in PEFT service, we believe GPU resource sharing will be increasingly important as GPUs become more powerful.

10 Conclusion

This paper proposes Collie, a system for co-serving parameter-efficient finetuning and inference of large language models. We observe the distinct GPU resource utilization features between finetuning and inference workloads and propose a holistic PEFT serving system that supports to co-serve finetuning requests without affecting inference latency.

References

- [1] Huggingface Models. <https://huggingface.co/models>, 2023. 9
- [2] NVIDIA MIG Partitioning Limitations and Resetting. <https://docs.nvidia.com/datacenter/tesla/mig-user-guide/index.html#partitioning>, 2023. 4
- [3] NVIDIA Multi-Instance GPU (MIG). <https://www.nvidia.com/en-us/technologies/multi-instance-gpu/>, 2023. 4
- [4] NVIDIA Multi-Process Service (MPS). <https://docs.nvidia.com/deploy/mps/index.html>, 2023. 4
- [5] Perlmutter supercomputer. <https://docs.nersc.gov/systems/perlmutter/architecture/>, 2023. 9
- [6] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming {Throughput-Latency} tradeoff in {LLM} inference with {Sarathi-Serve}. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 117–134, 2024. 9
- [7] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S Gulavani, and Ramachandran Ramjee. Sarathi: Efficient llm inference by piggy-backing decodes with chunked prefills. *arXiv preprint arXiv:2308.16369*, 2023. 10, 12
- [8] Ahsan Ali, Riccardo Pincioli, Feng Yan, and Evgenia Smirni. Optimizing inference serving on serverless platforms. *Proceedings of the VLDB Endowment*, 15(10), 2022. 11
- [9] Ebtesam Almazrouei, Hamza Alobeidli, Abdulaziz Alshamsi, Alessandro Cappelli, Ruxandra Cojocaru, Merouane Debbah, Etienne Goffinet, Daniel Heslow, Julien Launay, Quentin Malartic, Badreddine Noune, Baptiste Pannier, and Guilherme Penedo. Falcon-40B: an open large language model with state-of-the-art performance. 2023. 9
- [10] Zhihao Bai, Zhen Zhang, Yibo Zhu, and Xin Jin. {PipeSwitch}: Fast pipelined context switching for deep learning applications. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 499–514, 2020. 1, 12
- [11] Paul Barham, Aakanksha Chowdhery, Jeff Dean, Sanjay Ghemawat, Steven Hand, Daniel Hurt, Michael Isard, Hyeontaek Lim, Ruoming Pang, Sudip Roy, et al. Pathways: Asynchronous distributed dataflow for ml. *Proceedings of Machine Learning and Systems*, 4:430–449, 2022. 1
- [12] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020. 1, 2, 9
- [13] Lequn Chen, Zihao Ye, Yongji Wu, Danyang Zhuo, Luis Ceze, and Arvind Krishnamurthy. Punica: Multi-tenant lora serving. *Proceedings of Machine Learning and Systems*, 6:1–13, 2024. 1, 3, 12
- [14] Zhenqian Chen, Xinkui Zhao, Chen Zhi, and Jianwei Yin. Deepboot: Dynamic scheduling system for training and inference deep learning tasks in gpu cluster. *IEEE Transactions on Parallel and Distributed Systems*, 2023. 1
- [15] Seungbeom Choi, Sunho Lee, Yeonjae Kim, Jongse Park, Youngjin Kwon, and Jaehyuk Huh. Serving heterogeneous machine learning models on {Multi-GPU} servers with {Spatio-Temporal} sharing. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, pages 199–216, 2022. 2, 4, 12
- [16] Yujeong Choi and Minsoo Rhu. Prema: A predictive multi-task scheduling algorithm for preemptible neural processing units. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pages 220–233. IEEE, 2020. 12
- [17] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. Palm: Scaling language modeling with pathways. *arXiv preprint arXiv:2204.02311*, 2022. 1
- [18] Daniel Crankshaw, Xin Wang, Guilio Zhou, Michael J Franklin, Joseph E Gonzalez, and Ion Stoica. Clipper: A {Low-Latency} online prediction serving system. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, pages 613–627, 2017. 12
- [19] Databricks. Finetuning api in databricks mosaic ai training. <https://docs.mosaicml.com/projects/mcli/en/latest/finetuning/finetuning.html>, 2024. 1
- [20] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018. 2

- [21] Aditya Dhakal, Sameer G Kulkarni, and KK Ramakrishnan. Gslice: controlled spatial sharing of gpus for a scalable inference platform. In *Proceedings of the 11th ACM Symposium on Cloud Computing*, pages 492–506, 2020. 4, 12
- [22] Ning Ding, Yujia Qin, Guang Yang, Fuchao Wei, Zonghan Yang, Yusheng Su, Shengding Hu, Yulin Chen, Chi-Min Chan, Weize Chen, et al. Parameter-efficient fine-tuning of large-scale pre-trained language models. *Nature Machine Intelligence*, 5(3):220–235, 2023. 2
- [23] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020. 1
- [24] Luciano Floridi and Massimo Chiriatti. Gpt-3: Its nature, scope, limits, and consequences. *Minds and Machines*, 30:681–694, 2020. 1
- [25] Joshua Fried, Zhenyuan Ruan, Amy Ousterhout, and Adam Belay. Caladan: Mitigating interference at microsecond timescales. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 281–297, 2020. 2
- [26] Qiankun Gao, Chen Zhao, Yifan Sun, Teng Xi, Gang Zhang, Bernard Ghanem, and Jian Zhang. A unified continual learning framework with general parameter-efficient tuning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 11483–11493, 2023. 1
- [27] Wei Gao, Qinghao Hu, Zhisheng Ye, Peng Sun, Xiaolin Wang, Yingwei Luo, Tianwei Zhang, and Yonggang Wen. Deep learning workload scheduling in gpu datacenters: Taxonomy, challenges and vision. *arXiv preprint arXiv:2205.11913*, 2022. 1
- [28] Google. Fine-tuning with the gemini api. <https://ai.google.dev/gemini-api/docs/model-tuning>, 2024. 1
- [29] Arpan Gujarati, Reza Karimi, Safya Alzayat, Wei Hao, Antoine Kaufmann, Ymir Vigfusson, and Jonathan Mace. Serving DNNs like clockwork: Performance predictability from the bottom up. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 443–462. USENIX Association, November 2020. 11, 12
- [30] Jashwant Raj Gunasekaran, Cyan Subhra Mishra, Prashanth Thinakaran, Bikash Sharma, Mahmut Taylan Kandemir, and Chita R Das. Cocktail: A multidimensional optimization for model serving in cloud. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, pages 1041–1057, 2022. 12
- [31] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025. 1, 3
- [32] Junxian He, Chunting Zhou, Xuezhe Ma, Taylor Berg-Kirkpatrick, and Graham Neubig. Towards a unified view of parameter-efficient transfer learning. *arXiv preprint arXiv:2110.04366*, 2021. 1, 3
- [33] Robin J Hogan. Fast reverse-mode automatic differentiation using expression templates in c++. *ACM Transactions on Mathematical Software (TOMS)*, 40(4):1–16, 2014. 6
- [34] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for nlp. In *International Conference on Machine Learning*, pages 2790–2799. PMLR, 2019. 1
- [35] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021. 1, 2, 12
- [36] Nikoleta Iliakopoulou, Jovan Stojkovic, Chloe Alverti, Tianyin Xu, Hubertus Franke, and Josep Torrellas. Chameleon: Adaptive caching and scheduling for many-adapter llm inference environments. *arXiv preprint arXiv:2411.17741*, 2024. 1
- [37] Paras Jain, Ajay Jain, Aniruddha Nrusimha, Amir Gholami, Pieter Abbeel, Joseph Gonzalez, Kurt Keutzer, and Ion Stoica. Checkmate: Breaking the memory wall with optimal tensor rematerialization. In *Proceedings of Machine Learning and Systems 2020*, pages 497–511. 2020. 6
- [38] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014. 10
- [39] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with page-attention. In *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP '23*, page 611–626, New York, NY, USA, 2023. Association for Computing Machinery. 10, 12

- [40] Yunseong Lee, Alberto Scolari, Byung-Gon Chun, Marco Domenico Santambrogio, Markus Weimer, and Matteo Interlandi. {PRETZEL}: Opening the black box of machine learning prediction serving systems. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 611–626, 2018. 12
- [41] Brian Lester, Rami Al-Rfou, and Noah Constant. The power of scale for parameter-efficient prompt tuning. *arXiv preprint arXiv:2104.08691*, 2021. 1, 2
- [42] Jiamin Li, Hong Xu, Yibo Zhu, Zherui Liu, Chuanxiong Guo, and Cong Wang. Lyra: Elastic scheduling for deep learning clusters. In *Proceedings of the Eighteenth European Conference on Computer Systems*, pages 835–850, 2023. 12
- [43] Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. *arXiv preprint arXiv:2101.00190*, 2021. 1, 2
- [44] Xuying Li, Zhuo Li, Yuji Kosuga, and Victor Bian. Optimizing safe and aligned language generation: A multi-objective grpo approach, 2025. 1, 3
- [45] Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E Gonzalez, et al. {AlpaServe}: Statistical multiplexing with model parallelism for deep learning serving. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*, pages 663–679, 2023. 12
- [46] Haokun Liu, Derek Tam, Mohammed Muqeeth, Jay Mohata, Tenghao Huang, Mohit Bansal, and Colin A Raffel. Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning. *Advances in Neural Information Processing Systems*, 35:1950–1965, 2022. 3
- [47] Jiachang Liu, Dinghan Shen, Yizhe Zhang, Bill Dolan, Lawrence Carin, and Weizhu Chen. What makes good in-context examples for gpt-3? *arXiv preprint arXiv:2101.06804*, 2021. 1
- [48] Xiao Liu, Yanan Zheng, Zhengxiao Du, Ming Ding, Yujie Qian, Zhilin Yang, and Jie Tang. Gpt understands, too. *AI Open*, 2023. 2
- [49] Xinyue Liu, Harshita Didee, and Daphne Ippolito. Customizing large language model generation style using parameter-efficient finetuning. In *Proceedings of the 17th International Natural Language Generation Conference*, pages 412–426, 2024. 1
- [50] Sourab Mangrulkar, Sylvain Gugger, Lysandre Debut, Younes Belkada, Sayak Paul, and Benjamin Bossan. Peft: State-of-the-art parameter-efficient fine-tuning methods. <https://github.com/huggingface/peft>, 2022. 9
- [51] Yuning Mao, Lambert Mathias, Rui Hou, Amjad Almahairi, Hao Ma, Jiawei Han, Wen-tau Yih, and Madsian Khabsa. Unipelt: A unified framework for parameter-efficient language model tuning. *arXiv preprint arXiv:2110.07577*, 2021. 3
- [52] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Hongyi Jin, Tianqi Chen, and Zhihao Jia. Towards efficient generative large language model serving: A survey from algorithms to systems. *arXiv preprint arXiv:2312.15234*, 2023. 12
- [53] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, Chunan Shi, Zhuoming Chen, Daiyaan Arfeen, Reyna Abhyankar, and Zhihao Jia. Specinfer: Accelerating generative large language model serving with speculative inference and token tree verification, 2023. 9, 12
- [54] Xupeng Miao, Chunan Shi, Jiangfei Duan, Xiaoli Xi, Dahua Lin, Bin Cui, and Zhihao Jia. Spotserve: Serving generative large language models on preemptible instances. *ASPLOS*, 2024. 1, 11
- [55] Deepak Narayanan, Keshav Santhanam, Peter Henderson, Rishi Bommasani, Tony Lee, and Percy Liang. Cheaply estimating inference efficiency metrics for autoregressive transformer models. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. 9
- [56] Xiaonan Nie, Yi Liu, Fangcheng Fu, Jinbao Xue, Dian Jiao, Xupeng Miao, Yangyu Tao, and Bin Cui. Angelptm: A scalable and economical large-scale pre-training system in tencent. *Proceedings of the VLDB Endowment*, 16(12):3781–3794, 2023. 1
- [57] OpenAI. GPT-4 technical report. *CoRR*, abs/2303.08774, 2023. 1
- [58] OpenAI. Fine-tuning now available for gpt-4o. <https://openai.com/index/gpt-4o-fine-tuning/>, 2024. 1
- [59] Jonas Pfeiffer, Aishwarya Kamath, Andreas Rücklé, Kyunghyun Cho, and Iryna Gurevych. Adapterfusion: Non-destructive task composition for transfer learning. *arXiv preprint arXiv:2005.00247*, 2020. 1
- [60] Qwen, :, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jiahong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang,

- Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report, 2025. [9](#)
- [61] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019. [2](#)
- [62] Francisco Romero, Qian Li, Neeraja J Yadwadkar, and Christos Kozyrakis. {INFaaS}: Automated model-less inference serving. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*, pages 397–411, 2021. [2](#), [12](#)
- [63] Timo Schick and Hinrich Schütze. Exploiting cloze questions for few shot text classification and natural language inference. *arXiv preprint arXiv:2001.07676*, 2020. [2](#)
- [64] Haichen Shen, Lequn Chen, Yuchen Jin, Liangyu Zhao, Bingyu Kong, Matthai Philipose, Arvind Krishnamurthy, and Ravi Sundaram. Nexus: A gpu cluster engine for accelerating dnn-based video analysis. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, pages 322–337, 2019. [12](#)
- [65] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. page 1279–1297, 2025. [2](#)
- [66] Ying Sheng, Shiyi Cao, Dacheng Li, Coleman Hooper, Nicholas Lee, Shuo Yang, Christopher Chou, Banghua Zhu, Lianmin Zheng, Kurt Keutzer, et al. Slora: Scalable serving of thousands of lora adapters. *Proceedings of Machine Learning and Systems*, 6:296–311, 2024. [1](#), [3](#), [12](#)
- [67] Nisan Stiennon, Long Ouyang, Jeff Wu, Daniel M. Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul Christiano. Learning to summarize from human feedback, 2022. [3](#)
- [68] Nisan Stiennon, Long Ouyang, Jeffrey Wu, Daniel Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul F Christiano. Learning to summarize with human feedback. *Advances in Neural Information Processing Systems*, 33:3008–3021, 2020. [1](#)
- [69] Zhaoxuan Tan, Qingkai Zeng, Yijun Tian, Zheyuan Liu, Bing Yin, and Meng Jiang. Democratizing large language models via personalized parameter-efficient fine-tuning. *arXiv preprint arXiv:2402.04401*, 2024. [1](#)
- [70] MosaicML NLP Team. Introducing mpt-7b: A new standard for open-source, commercially usable llms, 2023. Accessed: 2023-05-05. [9](#)
- [71] NovaSky Team. Sky-t1: Train your own o1 preview model within 450. <https://novasky-ai.github.io/posts/sky-t1>, 2025. Accessed : 2025 – 01 – 09. [10](#)
- [72] ShareGPT Team, 2023. [10](#)
- [73] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023. [1](#), [9](#)
- [74] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models, 2023. [9](#)
- [75] Colin Unger, Zhihao Jia, Wei Wu, Sina Lin, Mandeep Baines, Carlos Efrain Quintero Narvaez, Vinay Ramakrishnaiah, Nirmal Prajapati, Patrick S. McCormick, Jamaludin Mohd-Yusof, Xi Luo, Dheevatsa Mudigere, Jongsoo Park, Misha Smelyanskiy, and Alex Aiken. Unity: Accelerating DNN training through joint optimization of algebraic transformations and parallelization. In *16th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2022, Carlsbad, CA, USA, July 11-13, 2022*, pages 267–284. USENIX Association, 2022. [5](#)
- [76] Yuxin Wang, Yuhao Chen, Zeyu Li, Xueze Kang, Zhenheng Tang, Xin He, Rui Guo, Xin Wang, Qiang Wang, Amelie Chi Zhou, and Xiaowen Chu. Burstgpt: A real-world workload dataset to optimize llm serving systems, 2024. [10](#)
- [77] Yuxin Wang, Yuhao Chen, Zeyu Li, Zhenheng Tang, Rui Guo, Xin Wang, Qiang Wang, Amelie Chi Zhou, and Xiaowen Chu. Towards efficient and reliable llm serving: A real-world workload study. *arXiv preprint arXiv:2401.17644*, 2024. [1](#)
- [78] Qizhen Weng, Wencong Xiao, Yinghao Yu, Wei Wang, Cheng Wang, Jian He, Yong Li, Liping Zhang, Wei Lin, and Yu Ding. {MLaaS} in the wild: Workload analysis and scheduling in {Large-Scale} heterogeneous {GPU} clusters. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, pages 945–960, 2022. [1](#)
- [79] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama

- Drame, Quentin Lhoest, and Alexander M. Rush. Transformers: State-of-the-art machine learning for pytorch, tensorflow, and jax. <https://github.com/huggingface/transformers>, 2022. 9
- [80] Bingyang Wu, Ruidong Zhu, Zili Zhang, Peng Sun, Xuanzhe Liu, and Xin Jin. {dLoRA}: Dynamically orchestrating requests and adapters for {LoRA}{LLM} serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 911–927, 2024. 1, 2, 3
- [81] Wencong Xiao, Romil Bhardwaj, Ramachandran Ramjee, Muthian Sivathanu, Nipun Kwatra, Zhenhua Han, Pratyush Patel, Xuan Peng, Hanyu Zhao, Quanlu Zhang, et al. Gandiva: Introspective cluster scheduling for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 595–610, 2018. 2, 3, 12
- [82] Wencong Xiao, Shiru Ren, Yong Li, Yang Zhang, Pengyang Hou, Zhi Li, Yihui Feng, Wei Lin, and Yangqing Jia. {AntMan}: Dynamic scaling on {GPU} clusters for deep learning. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 533–548, 2020. 3, 12
- [83] Fuxun Yu, Di Wang, Longfei Shangguan, Minjia Zhang, Chenchen Liu, and Xiang Chen. A survey of multi-tenant deep learning inference on gpu. *arXiv preprint arXiv:2203.09040*, 2022. 4
- [84] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for {Transformer-Based} generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 521–538, 2022. 8, 9
- [85] Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiaze Chen, Jiangjie Chen, Chengyi Wang, Hongli Yu, Weinan Dai, Yuxuan Song, Xiangpeng Wei, Hao Zhou, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Lin Yan, Mu Qiao, Yonghui Wu, and Mingxuan Wang. Dapo: An open-source llm reinforcement learning system at scale, 2025. 3
- [86] Biao Zhang, Ankur Bapna, Rico Sennrich, and Orhan Firat. Share or not? learning to schedule language-specific capacity for multilingual translation. In *Ninth International Conference on Learning Representations 2021*, 2021. 1
- [87] Chengliang Zhang, Minchen Yu, Wei Wang, and Feng Yan. {MArk}: Exploiting cloud services for {Cost-Effective},{SLO-Aware} machine learning inference serving. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, pages 1049–1062, 2019. 12
- [88] Haoyu Zhang, Jianjun Xu, and Ji Wang. Pretraining-based natural language generation for text summarization. *arXiv preprint arXiv:1902.09243*, 2019. 1
- [89] Hong Zhang, Yupeng Tang, Anurag Khandelwal, and Ion Stoica. SHEPHERD: Serving DNNs in the wild. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 787–808, Boston, MA, April 2023. USENIX Association. 11, 12
- [90] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, et al. Opt: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068*, 2022. 9
- [91] Justin Zhao, Timothy Wang, Wael Abid, Geoffrey Angus, Arnav Garg, Jeffery Kinnison, Alex Sherstinsky, Piero Molino, Travis Addair, and Devvret Rishi. Lora land: 310 fine-tuned llms that rival gpt-4, a technical report. *arXiv preprint arXiv:2405.00732*, 2024. 1
- [92] Yihao Zhao, Xin Liu, Shufan Liu, Xiang Li, Yibo Zhu, Gang Huang, Xuanzhe Liu, and Xin Jin. Muxflow: Efficient and safe gpu sharing in large-scale production deep learning clusters. *arXiv preprint arXiv:2303.13803*, 2023. 4
- [93] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yuanzhong Xu, Danyang Zhuo, Joseph E. Gonzalez, and Ion Stoica. Alpa: Automating inter- and intra-operator parallelism for distributed deep learning. *CoRR*, abs/2201.12023, 2022. 5
- [94] Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, Zheyang Luo, Zhangchi Feng, and Yongqiang Ma. Llamafactory: Unified efficient fine-tuning of 100+ language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, Bangkok, Thailand, 2024. Association for Computational Linguistics. 10
- [95] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving, 2024. 10
- [96] Han Zhou, Xingchen Wan, Ivan Vulić, and Anna Korhonen. Autopeft: Automatic configuration search for parameter-efficient fine-tuning. *arXiv preprint arXiv:2301.12132*, 2023. 3
- [97] Zhe Zhou, Xuechao Wei, Jiejing Zhang, and Guangyu Sun. {PetS}: A unified framework for {Parameter-Efficient} transformers serving. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, pages 489–504, 2022. 1, 3, 12
- [98] Barret Zoph and Quoc V Le. Neural architecture search with reinforcement learning. *arXiv preprint arXiv:1611.01578*, 2016. 3